

THE 4 PATTERNS THAT APPEAR IN EVERY PYKIT PROGRAM

1 IMPORT — LOAD YOUR TOOLS

```
import pykit_explorer      # always first
from neopixels import NeoPixels, Colors
from imu_sensor import IMUSensor
from digital_io import DigitalInput
```

Pattern: from <filename> import <ClassName>

Filename = the module (e.g. neopixels). ClassName = what you'll use (e.g. NeoPixels). Only import what your project needs.

2 CREATE — SET UP YOUR HARDWARE

```
# One line per piece of hardware
px = NeoPixels()
imu = IMUSensor()
btn = DigitalInput(board.D3)

# I2C sensors need a bus first
i2c = I2CBus()
sensor = BME680Sensor(i2c.bus)
```

Pattern: name = ClassName()

This is called *instantiation*. You only do it once at the top of your program. After this, px is your NeoPixels — forever.

3 CALL — MAKE THINGS HAPPEN

```
# Tell hardware what to do
px.fill(Colors.RED)      # all pixels red
px.off()                 # all pixels off
px.set(0, Colors.BLUE)   # pixel 0 = blue

# Ask hardware for data
direction = imu.tilt_direction() # "LEFT" / "RIGHT" etc
pressed = btn.is_pressed()      # True or False
```

Pattern: object.method()

The dot connects your variable to an action. Some calls *do* something (like turning on LEDs). Others *return* a value you can save and use.

4 LOOP + DECIDE — REACT TO THE WORLD

```
while True:              # run forever
    direction = imu.tilt_direction()
    if direction == "LEFT":
        px.fill(Colors.RED)
    elif direction == "RIGHT":
        px.fill(Colors.BLUE)
    else:
        px.off()
    time.sleep(0.05)      # short pause
```

Indentation matters. Everything inside while True; must be indented. Everything inside if / elif / else; must be indented again. Use 4 spaces.

F-STRINGS — DISPLAY DATA

```
temp = CPUTemperature()
c = temp.celsius

# Put a variable inside { }
print(f"Temp: {c} C")
print(f"Temp: {c:.1f} C")    # 1 decimal place

# Works with any variable
print(f"Tilt: {imu.tilt_direction()}")
```

Start the string with `f"..."` — then put any variable or expression inside `{ }`. The `:.1f` means "1 decimal place."

f-string is shorthand for *formatted string literal*.

COMPLETE 19-LINE PROGRAM

```
import pykit_explorer
from imu_sensor import IMUSensor
from neopixels import NeoPixels, Colors, OFF

imu = IMUSensor()
px = NeoPixels()

while True:
    direction = imu.tilt_direction()
    if direction == "LEFT":
        px.fill(Colors.RED)
    elif direction == "RIGHT":
        px.fill(Colors.BLUE)
    elif direction == "UP":
        px.fill(Colors.GREEN)
    elif direction == "DOWN":
        px.fill(Colors.YELLOW)
    else:
        px.off()
```

This is a complete, working program. It uses all 4 patterns. Tilt the board in any direction to change the NeoPixel colour.

MODULE QUICK PICK — WHAT DO YOU NEED?

INPUT	MODULE	CLASS	OUTPUT	MODULE	CLASS
BUTTON	digital_io	DigitalInput	RGB LEDS	neopixels	NeoPixels
TILT/MOTION	imu_sensor	IMUSensor	SCREEN	lcd_display	LCDDisplay
TOUCH PAD	cap_touch	CapTouch	SOUND/WAV	audio_out	AudioOutput
GESTURE	apds9960	APDS9960Sensor	SERVO MOTOR	servo_control	ServoController
ENV SENSOR	bme680	BME680Sensor	LED/BUZZER	pwm_out	PWMOutput
ANALOG PIN	analog_io	AnalogInput	LOG TO SD	sd_card	SDCard

PROJECT IDEAS — COMBINE THESE

PROJECT	COMBINE THESE MODULES
MOTION GAME	imu + neopixels + audio_out
GESTURE SOUND	apds9960 + audio_out
ENV DASHBOARD	bme680 + lcd_display + neopixels
THEREMIN	imu + apds9960 + audio_out
DATA LOGGER	bme680 + sd_card + lcd_display
WIRELESS	imu + bme680 + ble_uart

3 ERRORS YOU'LL SEE TODAY

IndentationError

Your spacing is off. Everything inside a while or if must be indented exactly 4 spaces. Don't mix tabs and spaces.

NameError: name 'px' is not defined

You used a variable before creating it. Make sure `px = NeoPixels()` appears before the `while True: loop`.

ImportError

The module file isn't on the CIRCUITPY drive, or you spelled it wrong. Check the /API folder matches your import statement.

BEFORE YOU START CODING — AM I READY?

PYTHON READINESS CHECK

- I can write from `neopixels import NeoPixels` without looking it up
- I know that `px = NeoPixels()` goes **before** `while True:`
- I understand that `if / elif / else` blocks must be indented
- I know `time.sleep(0.1)` pauses for 100 milliseconds
- I can read an error message and find the line number

PROJECT READINESS CHECK

- I know what **input** my project senses (button? tilt? gesture? proximity?)
- I know what **output** my project produces (LEDs? sound? screen? motor?)
- I've copied only the modules I need into the `/API` folder
- I've run the relevant minimal example from the README first
- I can open the serial console to read error messages

LEVEL UP — WHEN YOUR FIRST PROJECT IS DONE

DO TWO THINGS AT ONCE WITH ASYNCRUNNER

The problem with `time.sleep()` — while the board is sleeping, it can't do anything else. One blink blocks everything. `AsyncRunner` solves this with **cooperative multitasking**: each task runs a little, then *yields* to the others.

The two rules:

1. Your function must start with `async def`
2. Replace `time.sleep()` with `await AsyncRunner.sleep()`

When to reach for it: you want two or more things happening simultaneously — blinking LEDs while polling a sensor, playing audio while updating the display, running a timeout alongside user input.

```
import pykit_explorer
from neopixels import NeoPixels, Colors, OFF
from async_tasks import AsyncRunner

px = NeoPixels()

# async def — like a normal function, but pauseable
async def blink(pixel, interval, color):
    while True:
        px.set(pixel, color)
        await AsyncRunner.sleep(interval) # yield here
        px.set(pixel, OFF)
        await AsyncRunner.sleep(interval)

# All three run concurrently — none blocks the others
runner = AsyncRunner()
runner.add(blink(0, 0.30, Colors.RED))
runner.add(blink(1, 0.75, Colors.GREEN))
runner.add(blink(2, 1.00, Colors.BLUE))
runner.run() # starts all tasks
```

BLOCKING vs. CONCURRENT

```
# BEFORE — BLOCKING
while True:
    px.set(0, Colors.RED)
    time.sleep(0.5) # board frozen here
    px.set(0, OFF)
    time.sleep(0.5) # board frozen here
# pixel 1 never blinks — it's waiting
```

```
# AFTER — CONCURRENT
async def blink(pixel, interval, color):
    while True:
        px.set(pixel, color)
        await AsyncRunner.sleep(interval)
        px.set(pixel, OFF)
        await AsyncRunner.sleep(interval)
# all pixels blink independently
```

The mental model: think of each `async def` function as a separate worker. `await` is the moment a worker puts up their hand and says "I'm waiting — someone else can go." The runner keeps cycling through workers until everyone is done waiting, then repeats. No worker ever interrupts another mid-instruction — only at `await` points.